

```

container_name: open-webui
restart: unless-stopped
ports:
  - "3000:8080"
environment:
  - OLLAMA_BASE_URL=http://ollama:11434
volumes:
  - openwebui_data:/app/backend/data
depends_on:
  - ollama

volumes:
  ollama_data:
  openwebui_data:

docker compose up -d

```

You now have Ollama plus Open WebUI running together. Open WebUI gives your team a clean ChatGPT-style screen at port 3000.

I have used this exact setup in client demos more times than I can count. People see the familiar interface, realise their data stays inside their network, and the conversation immediately gets serious. It is almost predictable now.

Step 5: Build your own model with a Modelfile

This is where it starts feeling like proper engineering. Suppose you want a model that always behaves like a support bot for your product.

```

docker exec -it ollama bash
cat > /tmp/SupportBot << 'EOF'
FROM llama3.1:8b
PARAMETER temperature 0.3
PARAMETER num_ctx 8192
SYSTEM """
You are a polite, concise support assistant for TechCrave's
cloud platform.
Always provide step-by-step solutions. If you do not know an
answer,
say so honestly and suggest contacting human support.
"""
EOF
ollama create supportbot -f /tmp/SupportBot

```

Now you have a supportbot model. Push that Modelfile to Git. Treat it like any other config. Update it through the same review process you use for code.

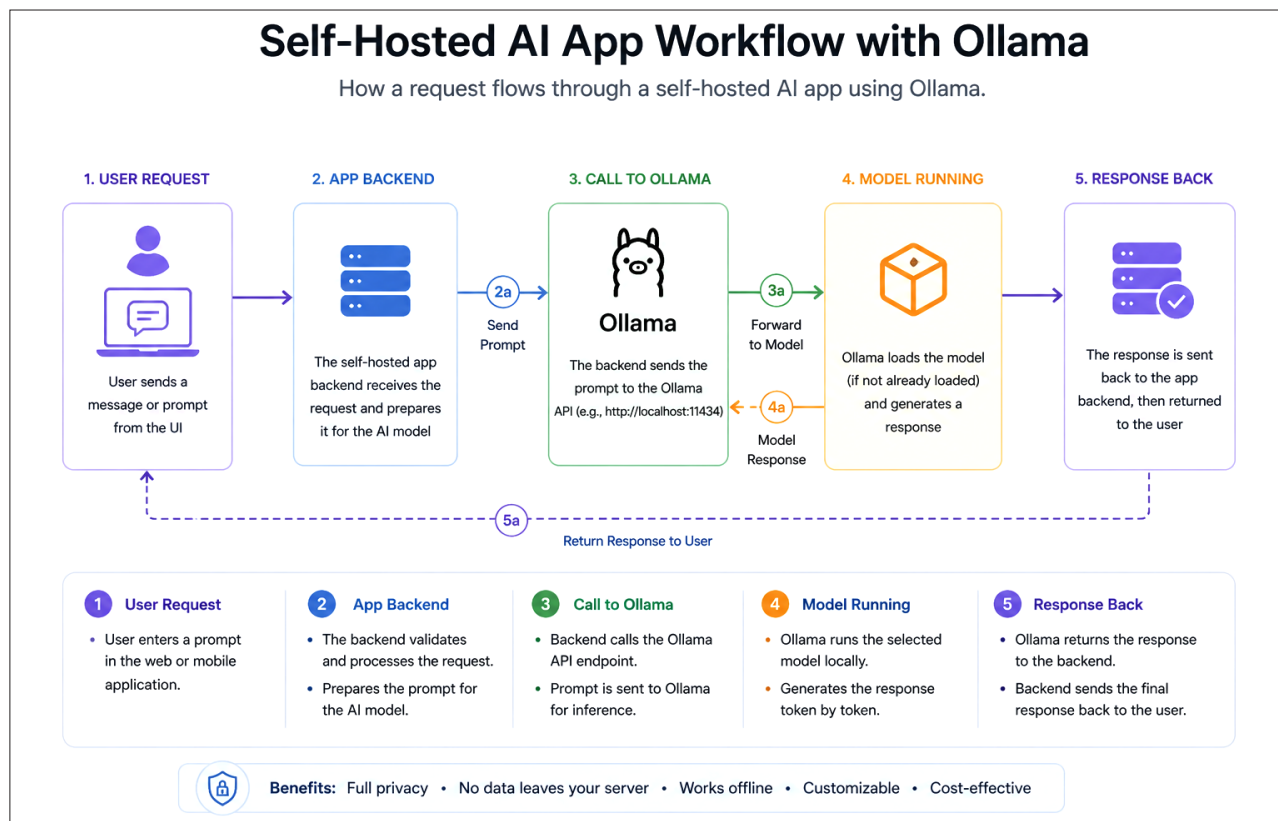


Figure 2: Self-hosted AI application